Technology of the Elbonian Peoples Republic
Edit Page
Share

Quiz JSON Schema Specification

This schema defines the structure for a multi-choice quiz application with dynamic section management, percentage-based topic distribution, and comprehensive question metadata.

Curated by maxwere
3 min read

Published Jul 26, 2025
6

Root Schema Structure

Top-Level Properties

Property	Type	Required	Description
`quiz_name`	string	✅	Display name of the quiz
`section_topics`	array	✅	Topic categories with percentage weights
`questions`	array	✅	Array of quiz questions

Detailed Field Specifications

1. quiz_name

Ask follow-up

- **Type**: `string`
- **Purpose**: Used for display in quiz headers, results pages, and navigation
- **Requirements**:

- Non-empty string
- Recommended length: 10-50 characters
- Used dynamically in UI elements

Example:

```
"quiz_name": "Saxon 4/5 Sample Exam"
```

2. section_topics

- **Type**: `array` of objects
- **Purpose**: Defines topic categories and their percentage distribution for dynamic section management

section_topics Object Schema:

Field	Type	Required	Constraints	Description
`name`	string	✅	1-100 characters	Section display integer

Validation Rules:

- All `percent` values must sum to exactly 100
- Section names must be unique within the array

Ask follow-up

```json
"section_topics": [
  { "name": "Number Sense & Operations", "percent": 30 },
  { "name": "Fractions & Decimals", "percent": 30 },
  { "name": "Geometry & Measurement", "percent": 20 },
  { "name": "Word Problems & Problem Solving", "percent": 20 }
]
```

3. questions

- **Type**: `array` of objects
- **Purpose**: Contains all quiz questions with metadata

Question Object Schema:

Field	Type	Required	Format	Description
`id`	integer	✅	Sequential starting from 1	Unique question identifier
`section`	string	✅	Must match a `section_topics.name`	Primary topic category
`subsection`	string	✅	1-100 characters	Specific subtopic within section
`question`	string	✅	10-500 characters	The question text
`choices`	array	✅	Exactly 4 strings	Multiple choice options
`correct_answer`	string	✅	Must match one choice exactly	Letter format (A, B, C, D)
`explanation`	string	✅	10-300 characters	Detailed answer explanation

Ask follow-up

- **Length**: Exactly 4 elements
- **Format**: Each choice prefixed with letter and parenthesis
- `"A) [option text]"`
- `"B) [option text]"`
- `"C) [option text]"`
- `"D) [option text]"`

Validation Rules for Questions:

- `id` values must be sequential integers starting from 1
- `section` must exactly match one of the `section_topics[].name` values
- `correct_answer` must be exactly "A", "B", "C", or "D"
- Question distribution should approximately match `section_topics` percentages

Example Question:

Ask follow-up

```json
{
  "id": 1,
  "section": "Number Sense & Operations",
  "subsection": "Addition & Subtraction",
  "question": "What is 527 + 289?",
  "choices": [
    "A) 806",
    "B) 816",
    "C) 716",
    "D) 826"
  ],
  "correct_answer": "B",
  "explanation": "527 + 289 = 816.",
  "source": "Saxon 4/5, Lesson 8"
}
```

Schema Constraints & Business Rules

Cross-Field Validation

1. **Section Alignment**: Every `questions[].section` must have a corresponding entry in `section_topics[].name`
2. **Percentage Distribution**: Actual question counts should approximate the `section_topics[].percent` distribution
3. **Answer Validation**: `correct_answer` must correspond to an existing choice (A-D)

Recommended Practices

Ask follow-up

- **Question Distribution**: For N questions, each section should contain approximately `(percent/100) × N` questions
- **ID Sequencing**: Question IDs should be consecutive integers (1, 2, 3, ...)
- **Content Quality**: Questions should be clear, unambiguous, and age-appropriate
- **Source Attribution**: Include specific lesson/chapter references for educational content

Data Integrity Checks

- Total percentage validation: `Σ(section_topics[].percent) = 100`
- Unique question IDs across the entire question array
- Non-empty required string fields
- Valid choice format with proper letter prefixes

JSON Schema (Technical)

Ask follow-up

```
json
```

Ask follow-up

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "required": ["quiz_name", "section_topics", "questions"],
  "properties": {
    "quiz_name": {
      "type": "string",
      "minLength": 1,
      "maxLength": 100
    },
    "section_topics": {
      "type": "array",
      "minItems": 1,
      "items": {
        "type": "object",
        "required": ["name", "percent"],
        "properties": {
          "name": {
            "type": "string",
            "minLength": 1,
            "maxLength": 100
          },
          "percent": {
```

Ask follow-up